SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5 **Total Marks:50**

Annexure A

SIMULATION TASK

Code of Conduct:

I M.V.Rakesh Reddy192524164/(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.V.Rakesh Reddy

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model

		independently			
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Annexure A

SIMULATION TASK

QUESTIONS

1. Given thethree-address code sequence:
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
Simulate the **peephole optimization** process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied, including algebraic simplification, constant folding, and dead code elimination.

2.Partition the following intermediate code into **basic blocks** and construct the corresponding **control flow graph**. Identify all leader statements and mention the rules used to identify them:
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2

3. Construct the **DAG** for the following basic block:
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
Using the DAG, identify all **common sub-expressions**. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

4. For a target machine with two registers **R0 and R1**, simulate the working of a **simple code generator** for the code:
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
Show the **register descriptor** and **address descriptor** after each instruction is processed.

5. For a **RISC-style target machine** with instructions **LOAD, STORE, ADD, SUB, MUL, MOV**, simulate target code generation for the expression:
(a - b) * (c + d) - e
Show the complete sequence of target instructions generated. Also explain the **runtime storage management** and **stack frame layout** used during the computation.
Given the three-address code sequence:
t1 = a + b,

t2 = t1 + 0,
t3 = t2 * 1,
if t3 == 0 goto L1,
t4 = t3 + t3

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – SIMULATION TASK

Course Code: CSA14 **Course Name:** Compiler Design

Course Outcome: CO5 – Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code.

Student Answer – Set of Solutions

The following solutions are written in a simple student-style format and follow the five questions given in the uploaded assessment sheet. The task focuses on peephole optimization, basic blocks, CFG, DAG, register descriptors, target-code generation and storage management. ■filecite■turn0file0■L112-L153■

Question 1 – Peephole Optimization

Given three-address code:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

Step 1: Algebraic simplification

Multiplication by 1 does not change a value. Therefore t1 = x * 1 can be replaced by t1 = x.

Subtraction of 0 does not change a value. Therefore t3 = t2 - 0 can be replaced by t3 = t2.

Multiplication by 0 always produces 0. Therefore t4 = t3 * 0 can be replaced by t4 = 0.

Step 2: Constant folding

After the above simplification, the conditional statement becomes:

```
if 0 != 0 goto L1
```

The condition is always false, so the jump can be removed.

Step 3: Dead code elimination

Since the jump is never taken, the temporary values t1, t2, t3 and t4 are not needed for the final computation. The remaining statement t5 = y + y is independent of them.

Optimized instruction sequence:

```
t5 = y + y
```

Thus, several unnecessary instructions are removed and the basic block becomes smaller and faster. The original question specifically asks for algebraic simplification, constant folding and dead code elimination. ■filecite■turn0file0■L114-L123■

Question 2 – Basic Blocks and Control Flow Graph

Given intermediate code:

```
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2
```

Leader identification rules

1. The first statement is always a leader.

2. Any statement that is the target of a conditional or unconditional jump is a leader.

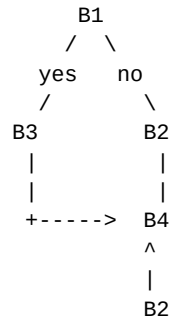
3. Any statement immediately following a jump statement is a leader.

Leaders

Statement 1 is a leader because it is the first statement. Statement 4 is a leader because it immediately follows the conditional jump at statement 3. Statement 6 is a leader because it is the target of the conditional jump. Statement 7 is a leader because it is the target of the unconditional jump at statement 5.

Basic Block	Statements
B1	1, 2, 3
B2	4, 5
B3	6
B4	7

Control Flow Graph



More precisely, B1 has two outgoing edges: B1 → B3 when y > 50, and B1 → B2 when the condition is false. B2 → B4 because statement 5 jumps to statement 7. B3 → B4 because statement 6 is followed by statement 7. Therefore B4 is the final block. The question and code sequence are given on page 3 of the assessment.
■filecite■turn0file0■L124-L132■

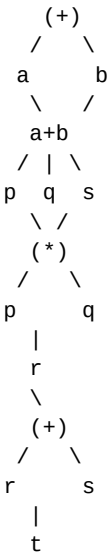
Question 3 – DAG and Common Sub-expressions

Given basic block:

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

DAG construction

The expression a + b occurs three times. In a DAG, the same operator node with the same operands is reused instead of creating three separate nodes.



Common sub-expression: a + b is the common sub-expression. It is calculated only once and its result can be used for p, q and s.

Optimized three-address code

```
t0 = a + b
r = t0 * t0
t = r + t0
```

Simple target code

```
LOAD R0, a
ADD R0, b
MOV R1, R0
MUL R0, R1
ADD R0, R1
STORE t, R0
```

Register allocation

R0 first stores the value of a + b. It is copied to R1 because the same value is required for both operands of the multiplication. R0 is then used for the multiplication result. R1 still contains a + b, so it can be added to the multiplication result without calculating a + b again. Finally, the result is stored in t.

This follows the assessment requirement to construct the DAG, find common sub-expressions and simulate a simple code generator with register decisions. ■filecite■turn0file0■L133-L140■

Question 4 – Register and Address Descriptors

Target machine has two registers R0 and R1. Code:

```
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
```

Assume initially both registers are empty. The address descriptor tells where the current value of each variable is available. The register descriptor tells which value is currently stored in each register.

After instruction	R0	R1	Important address information
Initial	empty	empty	a,b,c,d,e,f,t1,t2,t3,t4,t5 in memory when needed
t1 = a + b	t1	empty	t1 in R0
t2 = c * d	t1	t2	t1 in R0, t2 in R1
t3 = t1 - t2	t3	empty	t3 in R0; t2 no longer needed
t4 = e + f	t3	t4	t3 in R0, t4 in R1
t5 = t3 / t4	t5	empty	t5 in R0

Step-by-step target operations

```
LOAD R0, a
ADD R0, b
LOAD R1, c
MUL R1, d
SUB R0, R1
LOAD R1, e
ADD R1, f
DIV R0, R1
STORE t5, R0
```

At each step, a register is reused when its previous value is no longer required. This reduces unnecessary memory accesses and makes effective use of the two available registers. The assessment asks for the register descriptor and address descriptor after each instruction. ■filecite■turn0file0■L141-L148■

Question 5 – RISC Target Code and Storage Management

Expression:

```
(a - b) * (c + d) - e
```

Target code

```
LOAD R0, a
LOAD R1, b
SUB R0, R1
STORE T1, R0

LOAD R0, c
LOAD R1, d
ADD R0, R1
STORE T2, R0

LOAD R0, T1
LOAD R1, T2
MUL R0, R1
LOAD R1, e
SUB R0, R1
STORE result, R0
```

Explanation

First, a - b is calculated and stored in temporary location T1. Next, c + d is calculated and stored in T2. Both temporary results are loaded into registers and multiplied. Finally, e is loaded into the other register and subtracted from the multiplication result. The final value is stored in result.

Runtime storage management

During execution, registers are used for frequently accessed temporary values. When both registers are required for another operation, an intermediate value can be stored in a temporary memory location. T1 and T2 are therefore used as temporary storage locations.

Simple stack frame layout

```
Higher address
-------------------------
| Parameters / arguments |
-------------------------
| Return address         |
-------------------------
| Saved registers        |
-------------------------
| Local variables        |
-------------------------
| Temporary T1           |
-------------------------
| Temporary T2           |
-------------------------
Lower address
```

The stack frame contains the information needed during the procedure call. Temporary locations T1 and T2 hold intermediate expression results. Registers hold the active operands and results whenever possible. The assessment specifically asks for the complete target sequence and an explanation of runtime storage and stack-frame layout. ■filecite■turn0file0■L149-L153■

Additional Given Three-Address Sequence

```
t1 = a + b
t2 = t1 + 0
t3 = t2 * 1
if t3 == 0 goto L1
t4 = t3 + t3
```

The final lines on page 4 of the supplied assessment contain this additional sequence.
■filecite■turn0file0■L154-L160■

Optimization:

```
t1 = a + b
t2 = t1
t3 = t2
if t3 == 0 goto L1
t4 = t3 + t3
```

Here, t2 = t1 + 0 becomes t2 = t1 and t3 = t2 * 1 becomes t3 = t2. These are algebraic simplifications. Further optimization can use copy propagation to replace t3 by t1 in the condition and t4 computation when safe.

Conclusion

The five problems demonstrate the main CO5 concepts: peephole optimization, basic-block formation, control-flow graphs, DAG-based common sub-expression elimination, register and address descriptors, and RISC-style target code generation. These techniques reduce unnecessary operations and improve the generated code.